And that's just for the first example! Now, imagine that this spec file has grown a number of additional examples over time. Some of these definitions are not even going to be visible on your screen when you are looking at a failing example.

Consider the person who has to read through this code in six months when something breaks (it might be you!). The reader's first reaction at seeing those highlighted one-liner specs is, "What happened? What behavior are we testing here?" The TDD community calls this separation of cause and effect a *mystery guest*.[4]

The fare calculation is the core thing we're testing. It should be front and center in the specs.

That doesn't mean we need to repeat all that setup code, though. Recall that an RSpec example group is just a Ruby class. That means we can define helper methods on it, just as we would for any other class:

07-structuring-code-examples/06/transit/spec/berlin_transit_ticket_refactored_spec.rb

```
RSpec.describe BerlinTransitTicket do
  def fare_for(starting_station, ending_station)
    ticket = BerlinTransitTicket.new
    ticket.starting_station = starting_station
    ticket.ending_station   = ending_station
    ticket.fare
  end

  context 'when starting in zone A and ending in zone B' do
    it 'costs €2.70' do
      expect(fare_for('Bundestag', 'Leopoldplatz')).to eq 2.7
    end
  end

  context 'when starting in zone A and ending in zone C' do
    it 'costs €3.30' do
      expect(fare_for('Bundestag', 'Birkenwerder')).to eq 3.3
    end
  end
end
```

Now, it's explicit exactly what behavior we're testing, without our needing to repeat the details of the ticketing API. Someone reading your spec won't need to guess at implicit behavior; they'll see everything spelled out in black and white.

Moreover, we've gained a little flexibility. With the before hook, all our specs were required to run the hook just as it was written. With a helper method, we control the timing and content of the setup.

4. https://robots.thoughtbot.com/mystery-guest